

From terminal to Claude Code

Let's set up a CLI AI agent do a task for you

Orazio Angelini

What we'll do in 45 minutes

1. **Learn agents in five minutes** — what's under the hood: a language model, a harness, and a loop
2. **Agents in the terminal** — Get a sense of what the command line interface (CLI) is, and how to start your own agent
3. **The exercise** — run a clinic audit, and look at how it's configured
4. **Final thoughts** — what you take away

Everything you'll need to type or paste is in `copypaste.txt`, and a PDF of these slides is in the exercise folder — both inside the files we'll download in a bit.

Please interrupt me with questions / clarifications / if I talk too fast.
Slides are verbose (sorry!) so you can reuse them.

The chore we want to hand off

Every clinic, you check each patient against their **treatment target** - the goal of getting disease activity down to a low level.

- One patient at a time, by hand
- Tedious, and easy to slip on a busy day

By the end of this session, that whole job is **one sentence** typed into a computer.
And it will also have a dashboard to visualise the patients

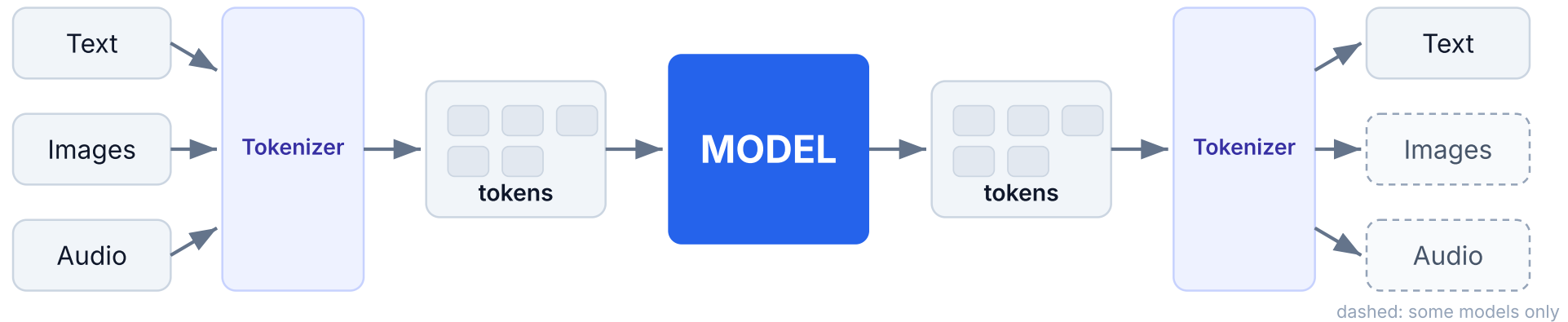
- Let's start with a 5-minutes overview of agents

What is an agent

- Think of an agent as someone who sits at your computer and can use it together with you.
- You can ask them to do things for you.
- Each agent has:
 - An LLM (brain)
 - A harness (body)
 - An agentic loop

The brain: a language model

A language model only predicts the next token.

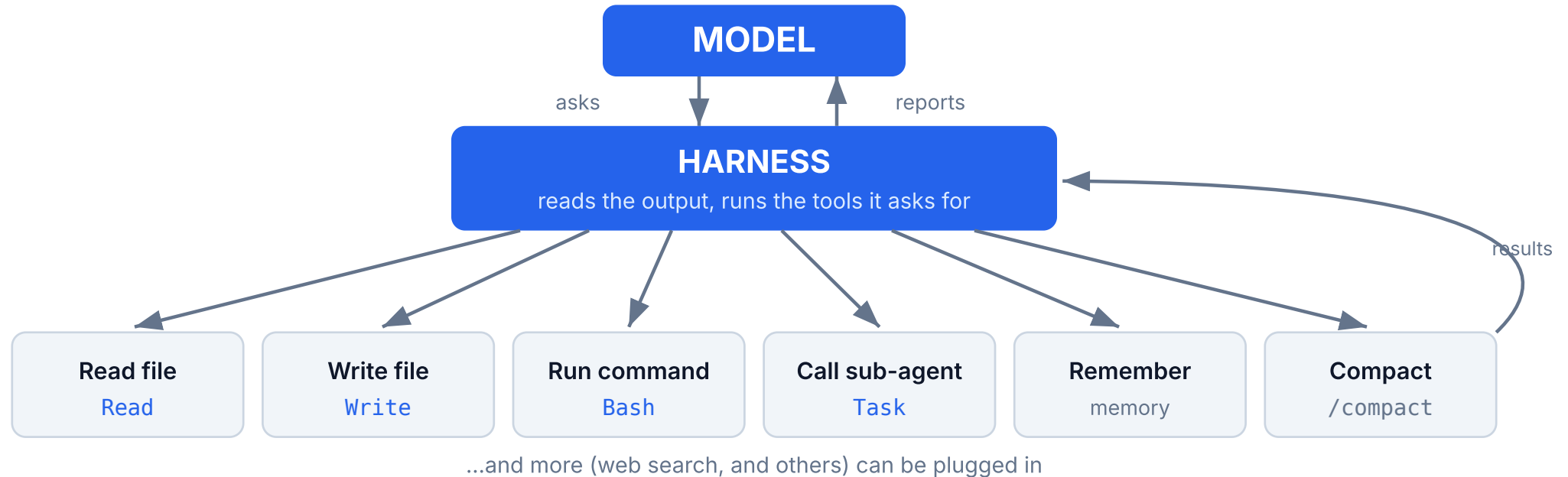


Brilliant at language, but only talk: no hands. A mind in a jar.

It can't reach your files: to read one, **you** paste it in; to *write* one, it tells you the text and **you** save it.

The body: a harness

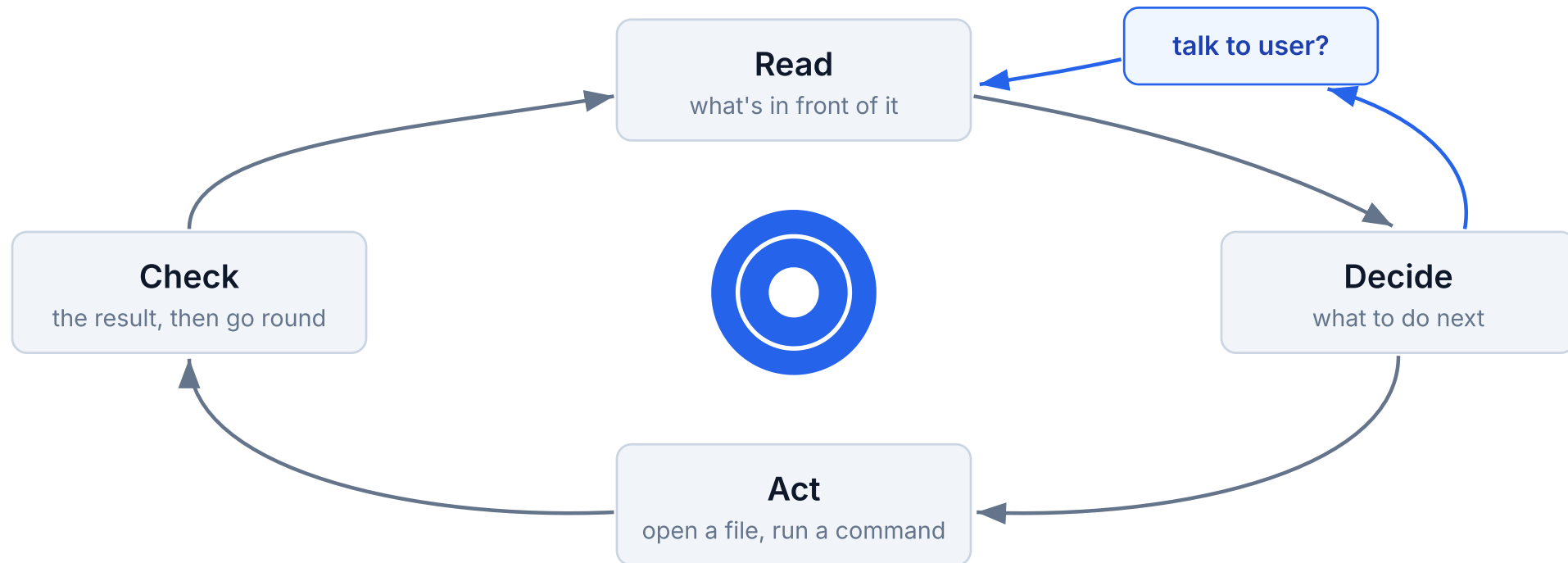
The harness turns the model's words into actions.



Claude Code is a harness. So are **OpenAI Codex** and free open-source tools.
Cursor (next session) is a harness with buttons instead of typed commands.

The agent: a loop

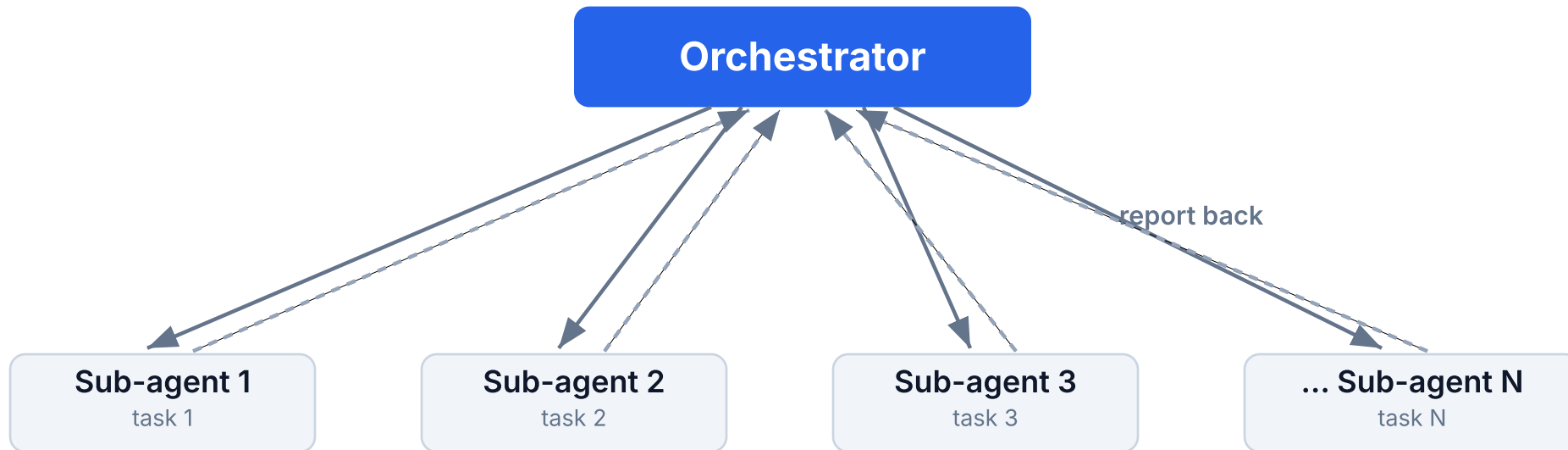
It keeps going round until the goal is met.



This loop, chasing a goal on its own, is an agent.

A swarm: many at once

One sub-agent per patient, all at once, reporting back.

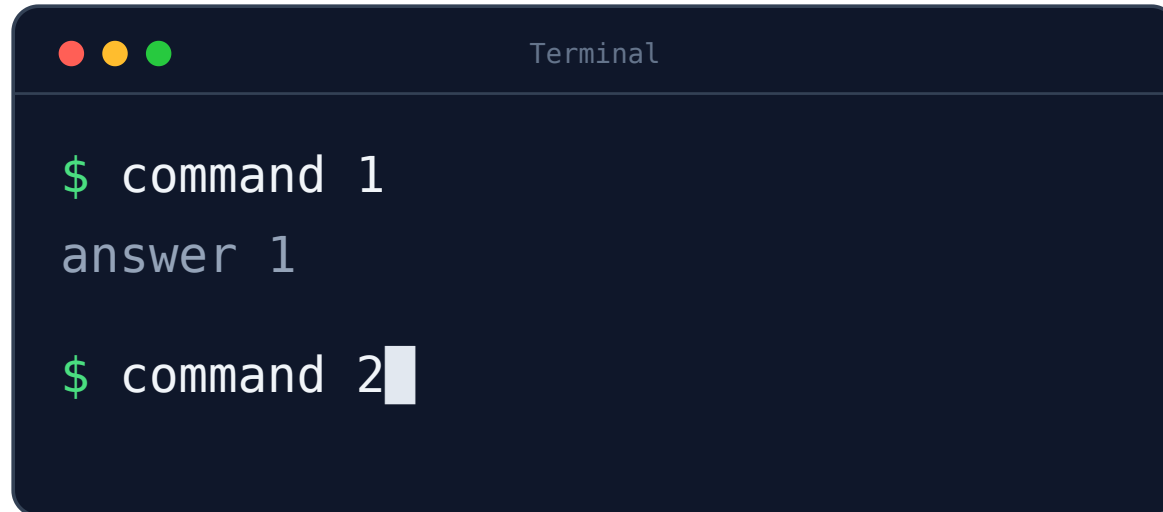


Each runs its own loop; you watch them side by side, like a board of jobs.

What is a terminal?

Before mice and windows, there was only *text* and keyboard. Old interaction mode: you type a command, it runs, the computer replies in text.

- A **direct, text way** to tell the computer what to do
- Everything you type is a command the computer executes
- One line in, one result out
- LLMs live inside text: most natural interface for them

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the word "Terminal". The terminal shows two commands being executed. The first command is "\$ command 1", followed by the output "answer 1". The second command is "\$ command 2", followed by a white cursor block.

```
Terminal

$ command 1
answer 1

$ command 2
```

Download the exercise

Get the patient letters and instructions from:

`github.com/ganileni/clinic-audit-exercise`

- Open that page, click the green **Code** button, choose **Download ZIP**
- **Unzip** it somewhere easy to find

Inside you'll find `copypaste.txt` (every command and link, ready to paste) and a **PDF of these slides to follow along**.

Comfortable with git? Instead: `git clone`

`https://github.com/ganileni/clinic-audit-exercise`

Open the terminal

- **Windows:** Start → type **PowerShell** → open
- **Mac:** Cmd+Space → type **Terminal** → open
- **Linux:** It depends (you should know if you're on Linux)

The few commands you need, the same in all three:

- `cd folder-name` — go into a folder
- `ls` — list what is in this folder
- `cd ..` — go back up one folder

A few more, harmless to try: `pwd` (which folder am I in?), `cat <file>` (show a file), `clear` (clean the screen), `date` (date and time).

Now `cd` into the folder you downloaded — but **you don't need to type the path**, copy it: **Windows** right-click the folder → *Copy as path* → Ctrl+V; **Mac** select it → Cmd+Option+C → Cmd+V (or drag it into the window).

Set up and launch

Do this in two terminals — we launch two agents (the audit and the dashboard), so set each one up the same way.

You did these in the setup session.

- **Install Claude Code** (if you haven't): go to `code.claude.com` — it shows the install command for your operating system (with a link to the docs just below)
- `export ANTHROPIC_API_KEY=<the key on the slide>` — your access pass. **don't do this if you have your own account already set up**
- `export CLAUDE_CODE_SUBAGENT_MODEL=haiku` — per-patient readers run on cheap **Haiku**
- `claude` — opens **Opus** by default, which directs the work
- Press **Shift+Tab** to let it run on its own (**auto mode**)

NOTE: Every command on this slide is in `copypaste.txt`.

Send the prompt

Paste this and **hit enter**. The run starts now and works in the background:

Follow the instructions in `AUDIT.md`. The patient letters are in `clinic/`. Write your output to `audit.json` in this directory.

That is the whole prompt. The rules, the output format, and the example all live in `AUDIT.md`.

It's better to write down long instructions in a file.

NOTE: Don't type it — this prompt is in `copypaste.txt`.

Send the dashboard prompt too

Now, in a new terminal, open a **second** agent and send this one, in parallel (remember to set it up and go into auto mode first):

Follow the instructions in `DASHBOARD.md` . Build `dashboard.html` ; it loads `audit.json` when the audit has written it.

Two jobs now run at once: the **audit** swarm, and the **dashboard** building alongside it.

This prompt is in `copypaste.txt` too.

They're running (about 8 minutes)

While the swarm audits and the dashboard builds, here is what we just asked

Why make a file called `AUDIT.md`?

The prompt is tiny. All the substance lives in one file, `AUDIT.md`.

- The prompt just says "follow `AUDIT.md`" (one sentence as I promised :))
- So the audit is **documented and reproducible**: anyone can read exactly what was asked
 - You can track changes and revert them (learn **version control software** and **git**)
- You can **ask an LLM to draft** the document. You just correct the mistakes.
- This is the beginning of a **skill**, a repeatable workflow.

Change the rule? Edit `AUDIT.md`, not a sentence you typed once and forgot.

Let's go through the document.

The document. The task and how we judge

The document begins with a description of the task, and then clearly introduces the definitions and the rule of the audit.

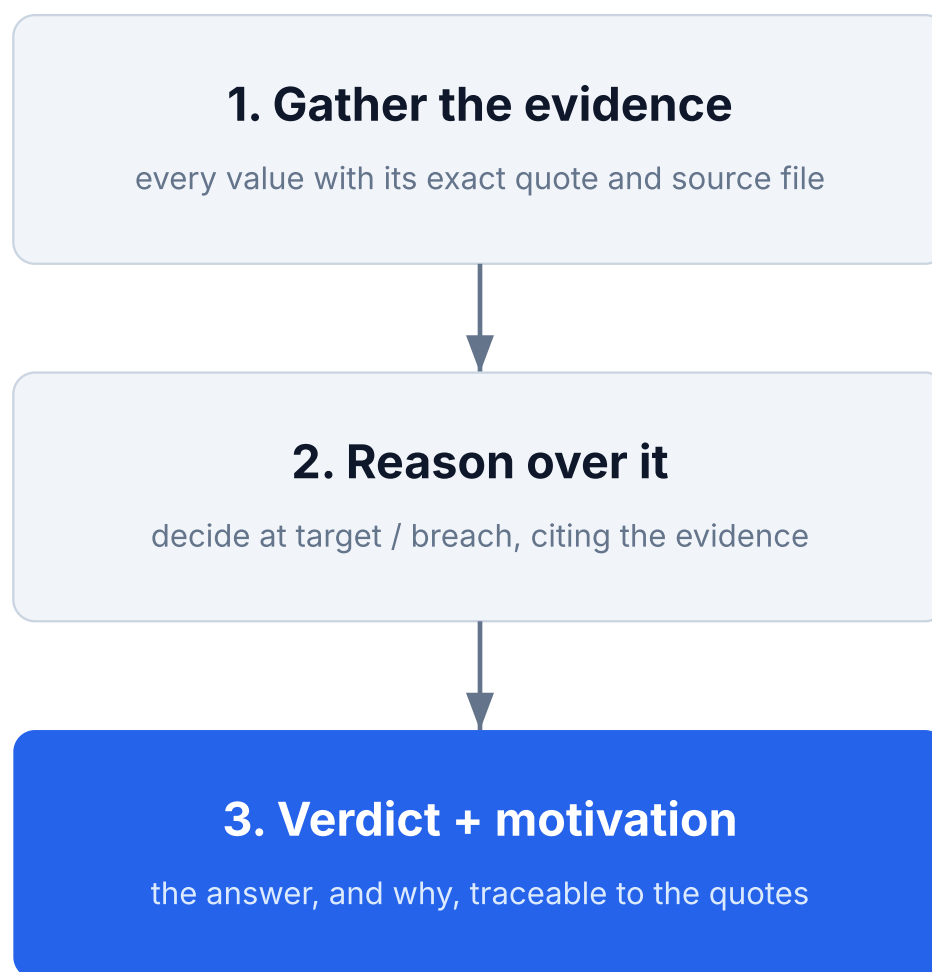
- **At target:** the disease activity score (DAS28-CRP) is **3.2 or below**
- A **breach** is all three together:
 - not at target, **and**
 - treatment was **not escalated**, **and**
 - **no reason** to hold off was written down
- Folic acid or a painkiller is **not** an escalation

Can't judge it cleanly? The agent flags **needs review**, instead of guessing

*A deliberately simplified teaching rule, not the full clinical standard. This needs to be iterated upon, and it's where **your expertise** comes in*

Prompting: elicited reasoning + grounding

A well-know technique to improve output



How it runs

For each patient, in order:

- **Gather** the records, which may be **split across several letters**
- One **sub-agent per patient**, all at once (the swarm)
- Keep the **exact quote and source** for every value

Watch for:

- Some letters give the raw numbers but **no score**
- The output is to **check, not trust**

What it delivers: a data file

The agent writes `audit.json`. *JSON* is just a simple, labelled data format that both people and computers can read.

- One entry per patient: the verdict, the reasoning, the evidence
- Every entry has the **same fixed shape** (a *schema*)

Why the *schema* fixed shape matters:

- Entries are **comparable** across patients
- A small **checker** can confirm nothing is missing or malformed
- Having the **schema** pre-specified, lets the dashboard agent know how the data looks like, so they can build a dashboard without ever having seen a piece of data.

Worked examples

Another proved method to improve output from an agent.

- Here we have 3,
- One is ambiguous one to reiterate that the agent should not force a decision under ambiguity (they are usually tempted to)
- You can add examples for weird and corner cases where agents got things wrong in the past!

Show and tell: improving the prompt

You don't have to write every rule yourself. **Ask the agent to change the instructions for you.**

- Live: I ask it to add one rule to `AUDIT.md` : **build a checker and run it** at the end, so the audit verifies its own output
- It edits the instructions; the next run follows them

Improve the recipe by talking to the agent, not just by typing.

It shows its sources

The audit has finished, and the dashboard we started earlier **fills with the results**: a **table of all patients** with their verdicts.

- Click a patient: the **reasoning**, and **every value with its quote and source letter**
- Click a source to open the **letter itself**
- A **review mode** steps through patients so you can **flag** the doubtful ones

Fast, but you still trace it back and check. Do not blindly trust the output.

Where this leaves you

You handed off a clinic audit in one sentence, with every answer **sourced**.

- The prompt needs to be iterated. Find mistakes, update the prompt, use it again. You will build trust (and a sense of what to check) as you go.
- You are responsible for the final output, always check it or rigorously demonstrate that the system performs comparably with you.

Dive deeper:

- Prompting: github.com/anthropics/prompt-eng-interactive-tutorial
- Claude Code guide: code.claude.com/docs/en/best-practices
- Power-user tricks: arps18.github.io/posts/claude-code-mastery

All three links are in `copypaste.txt` , ready to open.

What we learned today

Today we met **agents**, the **terminal**, and **how to start an agent**.

Things to remember:

- Write your instructions down — that is a **skill** (a reusable recipe)
- A little **prompting** craft goes a long way
- Use an **agent to help you write the prompt**
- **Markdown** and **JSON**: plain text both you and the agent can read. Also check out YAML.
- Make it **easy to check the result**

What to do now:

- Review the slides and the materials when you have time
- Use LLMs to dive deeper in the concepts we discussed. You can ask them infinite questions!
- Learn by doing: try your hand at a different task.

THANK YOU

Questions?